

Tol: Syntax Specification

Tomi Setsu

May 2, 2026

Abstract

This document describes the Tol surface syntax currently accepted by the compiler in this repository. It is derived directly from `tol/lexer.cpp`, `tol/ast-from-tokens.cpp`, `tol/pipe-lower-contract.cpp`, `tol/pipeline.h`, and the bundled standard library under `crypto/smartcont/tol-stdlib/`. The target compiler version is Tol 1.3.0.

Tol is TOS's own high-level smart-contract language: a typed, actor-oriented extension of FunC for TVM contracts. It keeps the low-level TVM control that FunC authors rely on, while adding automatic cell serialization, lazy loading, typed data layouts, `contract` blocks, typed `receive(...)` dispatch, deployment receivers, state machines, receiver-local query-id handling, and Slice 3–6 domain standard libraries.

This text is descriptive of the implementation as it exists today. Where the lexer reserves a token but the parser rejects or does not yet implement a surface form, that is stated explicitly.

Introduction

Tol is a statically typed source language for writing TOS smart contracts that compile to TVM/Fift-style assembly. It is designed as a high-level extension of FunC: the language keeps direct TVM/Fift-style escape hatches and explicit control over cells, slices, builders, continuations, and registers, while adding expressive typed data layouts, automatic serialization, lazy unpacking, union types, nullable types, generic functions and structs, methods, and a compiler-visible actor shape. Message dispatch, storage loading, state

transitions, getter methods, and several static-analysis checks are part of the language surface.

At the parser level, a Tol source file is a sequence of version declarations, imports, annotations, global declarations, constants, type aliases, functions, structs, enums, and at most one entrypoint **contract** declaration. The compiler automatically loads `@stdlib/common.tol` before the entrypoint file.

Contents

1	Overview	6
2	Relationship to FunC	6
3	Lexical Structure	7
3.1	Whitespace and Comments	7
3.2	Token Boundaries	8
3.3	Identifiers	8
3.4	Integer Literals	9
3.5	String Literals	9
3.6	Annotations	9
3.7	Reserved Keywords	10
4	Types	10
4.1	Type Expressions	10
4.2	Built-in and Standard Types	11
4.3	Nullable Types and Null Safety	11
4.4	Union Types and Smart Casts	12
4.5	Tensors, Arrays, and TVM Tuples	12
5	Top-Level Source Structure	13
5.1	Version Declaration	13
5.2	Imports	13
5.3	Global Variables	14
5.4	Constants	14
5.5	Type Aliases	14
6	Functions and Methods	15
6.1	Grammar	15
6.2	Methods	15
6.3	Entrypoint Names	16
6.4	Assembler and Builtin Bodies	16
6.5	Annotations on Functions	16
6.6	File-Scope Get Methods	17

7	Structs, Enums, and Serialization Prefixes	17
7.1	Struct Declarations	17
7.2	Serialization Prefixes	18
7.3	Struct Field Modifiers	18
7.4	Object Literals and Spread	19
7.5	Enums	19
8	Statements	20
8.1	Grammar	20
8.2	Local Variables	20
8.3	Throw, Assert, and Try/Catch	21
8.4	Unsupported Loop Control	21
9	Expressions	21
9.1	Primary Forms	21
9.2	Calls, Methods, and Postfix Operators	22
9.3	Casts and Type Tests	22
9.4	Match Expressions	23
9.5	Lambdas and Lazy Loading	23
9.6	Operator Precedence	23
10	Contract Blocks	24
10.1	Purpose	24
10.2	Contract Grammar	25
10.3	Typed Internal Receivers	26
10.4	Deployment Receivers	26
10.5	Unknown Opcode Handling	27
10.6	External Receivers	27
10.7	Get Methods	28
11	State Machines	28
11.1	State Declaration	28
11.2	Transitions	29
11.3	State Checks	29
12	Actor-Layer Static Analysis	30
12.1	Query-Id Propagation	30
12.2	Require Code Assignment	30

12.3 Storage and c4 Escape Restrictions	31
12.4 Postponement and Slice Checks	31
13 Standard Library Surface	31
13.1 Common Prelude	31
13.2 Explicit Imports	31
13.3 Slice-Layer Packages	32
14 Reserved but Unsupported Surface	33
A Summary Grammar	33
B Expression Precedence Summary	36
C Reference Sources	36

1 Overview

The current Tol frontend has five relevant layers:

1. lexical analysis, implemented in `tol/lexer.cpp`;
2. AST construction, implemented in `tol/ast-from-tokens.cpp`;
3. symbol registration, type resolution, type inference, lvalue/rvalue checking, pure/impure checking, and borrow checking;
4. TOS actor-layer checks and lowering, including state-machine, field-scoping, receive-exhaustiveness, contract lowering, require-code assignment, query-id propagation, reply correlation, and postponement checks;
5. code generation to FIF output.

The compilation pipeline is ordered deliberately. In the current implementation the contract checks run before `pipeline_lower_contracts()`, while `pipeline_assign_require_codes()`, `pipeline_check_query_id_propagation()`, `pipeline_check_slice3_reply_correlation()`, and `pipeline_check_postponement()` run after contract lowering, when the synthesized entrypoint bodies are visible.

2 Relationship to FunC

FunC exposes the TVM programming model directly: authors commonly write entrypoint dispatch by hand, read an opcode from the inbound body, decode the selected message, mutate persistent data in `c4`, and construct outbound actions explicitly. Tol is the higher-level TOS contract language built on that model. It keeps the hand-written entrypoint style available:

```
struct (0x12345678) CounterIncrement {
    incBy: uint32
}

type AllowedMessage = CounterIncrement | CounterReset

fun onInternalMessage(in: InMessage) {
```

```
val msg = lazy AllowedMessage.fromSlice(in.body);
match (msg) {
  CounterIncrement => { ... }
  CounterReset => { ... }
  else => { ... }
}
}
```

A file may define `fun onInternalMessage(in: InMessage)` directly, provided it does not also declare a `contract` block. The same actor shape can be written as compiler-visible Tol contract syntax:

```
contract Counter {
  storage: CounterStorage

  @disclaim_query_id
  receive(msg: CounterIncrement) {
    save(CounterStorage { value: storage.value + msg.incBy });
  }
}
```

The contract surface is more than a shorter spelling for manual dispatch. It gives later compiler passes a closed set of receiver types and states, which enables receive exhaustiveness, state reachability, field scoping, query-id propagation, reply-correlation checks, and actor-layer hardening before code generation.

3 Lexical Structure

3.1 Whitespace and Comments

Whitespace separates tokens and is otherwise ignored. The lexer recognizes:

- end-of-line comments beginning with `//`;
- block comments delimited by `/*` and `*/`.

Block comments are not nested. If a block comment reaches the end of the file without a closing delimiter, the lexer treats the rest of the file as comment text.

3.2 Token Boundaries

The lexer is implemented as a longest-prefix token trie. The following single-character tokens are always token boundaries:

+ - * / % ? : , ; () [] { } =
< > ! & | ^ ~ .

The lexer also recognizes the multi-character tokens:

... == != <= >= <=> << >> ~>> ^>>
&& || ~/ ^/
+= -= *= /= %= &= |= ^= <<= >>=
-> => ++ -- ??

The tokens ++ and -- are recognized only to produce a targeted compile-time error. Tol has no increment or decrement operator.

3.3 Identifiers

Ordinary identifiers begin with an ASCII letter, underscore, dollar sign, or a backtick-delimited identifier. After the first character, ordinary identifiers may contain ASCII letters, digits, underscores, or dollar signs:

```
counter
queryId
$generated
_privateName
```

Backticks may wrap identifiers, including identifiers that would otherwise be reserved words:

```
fun 'do'(): int { return 0; }
var 'with spaces' = 1;
```

Backtick identifiers end at the next backtick before a newline. An unclosed backtick is a lexical error.

3.4 Integer Literals

Integer literals are parsed as signed 257-bit values after tokenization. The lexer accepts:

- decimal literals, for example 0, 17, 1_000_000_000;
- hexadecimal literals with lowercase 0x, for example 0xff and 0x3b_9a_ca_00;
- binary literals with lowercase 0b, for example 0b0011_1011.

Hexadecimal digits may be uppercase or lowercase, but the prefix itself is lowercase. Underscore separators are allowed only between digits. A separator immediately after 0x or 0b is rejected by the parser. Negative literals are not separate lexical tokens; -1 is parsed as unary minus applied to 1 and then folded into a negative integer constant during AST construction.

3.5 String Literals

Strings are delimited by ordinary double quotes or by triple double quotes. Ordinary strings may not span lines. Triple-quoted strings may span lines and are mainly used for long assembler bodies and codegen tests.

The parser unescapes:

`\n \r \t \\ \' \"`

No FunC-style string suffixes are accepted. Compile-time string operations such as CRC and SHA calculations are exposed through standard-library functions and methods rather than through literal postfixes.

3.6 Annotations

An annotation token begins with @ and may contain alphanumeric characters, underscores, and dots. The compiler gives special meaning to:

```
@pure @inline @inline_ref @noinline @method_id
@overflow1023_policy @on_bounced_policy @on
@deploy @disclaim_query_id @initial
@unknown_silent_drop @unknown_throw
@implicit_protocol_default @implicit_protocol_for
```

The parser also allows opaque custom annotations of the forms `@custom`, `@custom.*`, `@test`, `@test.*`, and `@deprecated`. Unknown non-custom annotations are rejected. Several of the contract annotations above are accepted only in the specific contract member positions described in Section 10.

3.7 Reserved Keywords

The lexer recognizes the following keywords:

```
fun contract receive receive_external storage
type enum struct operator infix
global const var val mutate self
asm builtin private readonly
true false null
return repeat do while break continue
try catch throw assert if else match lazy
as is tol import export
```

Some keywords, such as `operator`, `infix`, and `export`, are reserved but currently rejected as unsupported top-level declarations.

4 Types

4.1 Type Expressions

The parser uses the following grammar for type expressions:

```
TypeExpr ::= [ "|" ] NullableType { "|" NullableType } [ "->" TypeExpr ]

NullableType ::= SimpleType [ "<" TypeExpr { "," TypeExpr } [ "," ] ">" ]
               [ "?" ]

SimpleType ::= identifier
            | "self"
            | "contract" | "receive" | "receive_external" | "storage"
            | "null"
            | "(" [ TypeExpr { "," TypeExpr } [ "," ] ] ")"
            | "[" [ TypeExpr { "," TypeExpr } [ "," ] ] "]"
```

The leading `|` in a union is accepted, mirroring TypeScript-style union notation. Function types are right-associative.

Examples:

```
int
int?
int | slice | null
Cell<MyStruct>
map<uint256, coins>
(int, slice)
[int, slice]
(int, slice) -> bool
int -> int -> int
```

4.2 Built-in and Standard Types

The built-in type names are declared in `@stdlib/common.tol`, which is auto-imported. The main primitive and standard-library-backed types are:

```
int bool cell slice builder continuation
address any_address never unknown void
intN uintN coins
varint16 varuint16 varint32 varuint32
string bitsN bytesN
map<K,V> array<T> lisp_list<T> dict tuple Cell<T>
```

Most primitive types occupy one TVM stack slot. Tensors occupy as many stack slots as they have components. Structs are named tensor-like values at the language level, but they may serialize into cells, builders, or slices through generated pack/unpack code.

4.3 Nullable Types and Null Safety

The suffix `?` forms a nullable type:

```
int?
Cell<Payload>?
address?
```

The expression `null` has the null type. The postfix `!` asserts non-nullness. Equality and inequality comparisons against `null` are rewritten into type checks internally, so `x == null` and `x != null` participate in smart casts.

4.4 Union Types and Smart Casts

Union types are written with `|`:

```
type AllowedMessage = Increment | Reset | null
```

The operators `is` and `!is` test a value against a type and feed the smart-cast analysis:

```
if (msg is Increment) {  
    return msg.incBy;  
}
```

4.5 Tensors, Arrays, and TVM Tuples

Tol distinguishes:

- tensors, written `(T1, T2, ...)`, which occupy several stack slots;
- arrays, written `array<T>` and commonly constructed with `[ ... ]`, which are backed by a TVM tuple value;
- the raw `tuple` type, an alias for `array<unknown>`.

Tensor components and array-like shaped values may be accessed with dot-index syntax:

```
var pair = (1, 2);  
pair.1 = 3;
```

5 Top-Level Source Structure

A source file is parsed as:

```
SourceItem ::= TolVersion
              | Import
              | Annotation
              | GlobalDecl
              | ConstDecl
              | TypeAlias
              | FunctionDecl
              | StructDecl
              | EnumDecl
              | ContractDecl
              | TopLevelGetter
              | ";"
```

```
Source ::= { SourceItem }
```

Annotations are queued and then consumed by the following declaration. A trailing annotation with no declaration is rejected.

5.1 Version Declaration

The version declaration is:

```
tol 1.3
tol 1.3.0
```

Only exact version comparison is implemented. If the declared version differs from the compiler version, the compiler emits a warning rather than rejecting the source.

5.2 Imports

Imports have the form:

```
import "relative-file.tol"
import "@stdlib/dicts"
```

Relative imports are resolved against the current file's directory. Imports whose text starts with `@` are resolved through configured path mappings; `@stdlib/...` is the standard-library mapping. Imported files are parsed once per real path.

5.3 Global Variables

Global variable declarations have the form:

```
global name: Type
```

Multiple global declarations in one statement are rejected. Assigning to a global at declaration time is also rejected.

5.4 Constants

Constants have the form:

```
const NAME = expr  
const NAME: Type = expr
```

The initializer is parsed as an ordinary expression and later checked as a constant expression by `pipeline_check_constant_expressions()`. Multiple constant declarations in one statement are rejected.

5.5 Type Aliases

Type aliases have the form:

```
type UserId = uint256  
type Result<T, E = int> = Ok<T> | Err<E>
```

The special form `type map<K,V> = builtin` is used by the standard library to declare compiler-backed types.

6 Functions and Methods

6.1 Grammar

The function grammar is:

```
FunctionDecl ::= "fun" [ ReceiverType "." ] name [ Generics ]  
               ParameterList [ ":" ReturnType ]  
               ( Block | AsmBody | "builtin" )  
  
AsmBody ::= "asm" [ "(" { parameter-name } [ "->" { integer-literal } ] ")" ]  
           string-literal { string-literal }  
  
Generics ::= "<" GenericItem { "," GenericItem } [ "," ] ">"  
GenericItem ::= identifier [ "=" TypeExpr ]  
  
ParameterList ::= "(" [ Parameter { "," Parameter } [ "," ] ] ")"  
Parameter ::= [ "mutate" ] (identifier | "self" | "_")  
             [ ":" TypeExpr ] [ "=" Expr ]
```

Parameter types are mandatory for ordinary functions except for the special `self` parameter of methods. Lambda parameters may omit types. `mutate` parameters cannot have default values.

If a return type is omitted, the compiler infers it. Omitted return type does not mean `void`.

6.2 Methods

A function whose name is preceded by a receiver type and a dot is a method:

```
fun Point.sum(self): int {  
    return self.x + self.y;  
}  
  
fun builder.storePair(mutate self, x: int, y: int): void {  
    self.storeInt(x, 32);  
    self.storeInt(y, 32);  
}
```

The first parameter may be `self`. Without `mutate`, `self` is immutable. A method may declare : `self` as its return type only when it accepts `self` as its first parameter.

6.3 Entrypoint Names

The following top-level function names are reserved entrypoints when they have no receiver:

```
main onInternalMessage onExternalMessage onRunTickTock
onSplitPrepare onSplitInstall onBouncedMessage
```

FunC/Fift-style names such as `recv_internal` and `recv_external` are rejected with a diagnostic telling the author to use `onInternalMessage`.

6.4 Assembler and Builtin Bodies

An assembler function has the form:

```
fun f(x: int): int asm "INC";
fun g(a: int, b: int): (int, int) asm(a b -> 1 0) "SWAP";
```

An `asm` function must specify a return type. The optional order clause lists argument names and returned stack indexes. One or more string literals form the assembler command body. The parser rejects assembler functions with more than 16 formal parameters.

The `builtin` body form is reserved for compiler-backed functions in the standard library:

```
fun slice.loadInt(mutate self, len: int): int
    builtin
```

6.5 Annotations on Functions

The function annotations accepted by the parser are:

- `@pure;`
- `@inline;`

- `@inline_ref`;
- `@noinline`;
- `@method_id(expr)` on regular top-level functions, where `expr` must resolve to an integer method id or a string method name;
- `@on_bounced_policy("manual")` or `@on_bounced_policy("ignore")` on `onInternalMessage`.

Opaque custom annotations are allowed but ignored by the compiler.

6.6 File-Scope Get Methods

At top level, `get fun name(...)` declares a legacy file-scope get method:

```
get fun seqno(): uint32 {
    return 0;
}
```

File-scope getters must be ordinary code-block functions: no receiver type, no generic parameters, no `mutate` parameters, and no `asm` or `builtin` body. A file containing a `contract` block cannot also declare a file-scope `get fun`.

7 Structs, Enums, and Serialization Prefixes

7.1 Struct Declarations

Struct declarations have the form:

```
struct Name [ Generics ] [ StructBody ]
struct (PrefixExpr) Name [ Generics ] StructBody

StructBody ::= "{" { StructField [ "," | ";" ] } "}"
StructField ::= [ FieldAnnotations ] [ "private" ] [ "readonly" ]
               name ":" TypeExpr [ FieldAnnotations ] [ "=" Expr ]
```

If the body is omitted, the struct is empty.

A struct declaration may carry `@overflow1023_policy("suppress")`. This suppresses the static warning that auto-serialization can theoretically exceed the TVM cell limits of 1023 bits or four references. The annotation does not change the wire layout; an actually oversized value can still fail at runtime.

7.2 Serialization Prefixes

The optional expression in `struct (PrefixExpr) Name` declares a serialization prefix. For ordinary TOS messages, a 32-bit prefix is used as the message opcode:

```
const OP_TRANSFER: uint32 = 0x0f8a7ea5

struct (OP_TRANSFER) Transfer {
  queryId: uint64
  amount: coins
}
```

Tol permits prefixes that are not 32 bits for general serialization and union encoding. TOS `contract receive(...)` declarations are stricter: every received message type must resolve to exactly a 32-bit prefix.

7.3 Struct Field Modifiers

Fields may be `private` and `readonly`:

```
struct Cursor {
  private readonly t: tuple
  index: uint16 = 0
}
```

`private` fields are accessible only through methods on that struct. `readonly` fields cannot be modified after construction.

State-bearing contracts may also use `@on(State1, State2)` to restrict a storage field to a set of states:

```
struct AuctionStorage {  
    winner: address @on(Closed)  
}
```

The `@on` annotation may appear before the field or after its type. It is valid only on fields of the storage struct of a state-bearing contract. Field default expressions in non-generic structs are checked as compile-time constant expressions.

7.4 Object Literals and Spread

Object literals are written either with an explicit type or with an inferred type:

```
val p1: Point = { x: 1, y: 2 };  
val p2 = Point { x: 1, y: 2 };  
val p3 = { ...p2, y: 3 };
```

The shorthand field syntax `{ x, y }` expands to `{ x: x, y: y }`.

7.5 Enums

Enum declarations have the form:

```
enum Color {  
    Red,  
    Green,  
    Blue = 7,  
}  
  
enum ErrorClass: uint8 {  
    Ok = 0,  
    Protocol = 4,  
}
```

Members may omit explicit values. The optional colon type fixes the serialized integer type and must resolve to an integer-width type such as `uint8` or to `coins`.

8 Statements

8.1 Grammar

Function and receiver bodies contain:

```
Stmt ::= LocalVarDecl
      | Block
      | "return" [ Expr ]
      | "if" "(" Expr ")" Block [ "else" (IfStmt | Block) ]
      | "repeat" "(" Expr ")" Block
      | "while" "(" Expr ")" Block
      | "do" Block "while" "(" Expr ")"
      | "throw" Expr
      | "throw" "(" Expr [ "," Expr ] ")"
      | "assert" "(" Expr "," Expr ")"
      | "assert" "(" Expr ")" "throw" Expr
      | "try" Block "catch" [ "(" CatchVars ")" ] Block
      | Expr
      | ";"
```

Inside contract receivers, the pseudo-statements `become` `State` and `keep_state` are also recognized.

Within a block, statements that do not end with a braced statement require a semicolon separator. A final statement before `}` may omit the semicolon when the parser can see the closing brace.

8.2 Local Variables

Local variables use `var` or `val`:

```
var x = 1;
val y: uint32 = 2;
var (a, b) = (1, 2);
var [first, second] = [1, 2];
var late: cell;
```

`val` bindings are immutable. A local declaration without an initializer is accepted only in the late-init form with an explicit type.

8.3 Throw, Assert, and Try/Catch

`throw` accepts either an exit code or an exit code plus argument:

```
throw 0xffff;
throw (0xffff, payload);
```

`assert` accepts either a comma form or a FunC-like `throw` form:

```
assert(x > 0, 100);
assert(x > 0) throw 100;
```

A `catch` clause may bind no variables, one variable, or two variables:

```
try { ... } catch { ... }
try { ... } catch (excNo) { ... }
try { ... } catch (excNo, arg) { ... }
```

8.4 Unsupported Loop Control

The lexer reserves `break` and `continue`, but the parser currently rejects both with a diagnostic. Loop bodies must be structured without them.

9 Expressions

9.1 Primary Forms

The highest-precedence expression forms are:

```
Primary ::= "("
          | "(" Expr ")"
          | "(" Expr "," Expr { "," Expr } [ "," ] ")"
          | "[" [ Expr { "," Expr } [ "," ] ] "]"
          | integer-literal
          | string-literal
          | "_"
          | "true" | "false" | "null" | "self"
          | identifier [ InstantiationTypes ]
          | TypeName ObjectBody
```

```
| TypeName "[" ... "]"
| ObjectBody
| MatchExpr
| LambdaExpr
| LazyExpr
```

The empty parenthesized expression `()` is the unit tensor. Square brackets construct an array-like value backed by a TVM tuple. An object body without an explicit type relies on contextual type inference.

9.2 Calls, Methods, and Postfix Operators

Function calls and method calls are left-associative:

```
f(1, 2)
builder.storeUint(1, 32).storeUint(2, 32)
genericFunction<int>(x)
obj.method<uint32>(arg)
```

Arguments may be marked `mutate` when the callee declares the corresponding parameter `mutate`:

```
state.receiveBid(mutate queue, bid);
```

The postfix non-null assertion is `!`:

```
value!.field
```

9.3 Casts and Type Tests

The cast operator is `as`:

```
val small = x as uint32;
```

Type tests use `is` and `!is`:

```
if (msg is Transfer) { ... }
if (msg !is Transfer) { ... }
```

9.4 Match Expressions

`match` is an expression:

```
val result = match (subject) {  
    Transfer => subject.amount,  
    Reset => 0,  
    else => throw 0xffff,  
};
```

Each arm pattern is parsed either as a type, as a constant expression, or as `else`. Unbraced expression arms require comma separation; braced arms may omit the comma.

The subject may also be a local declaration:

```
match (var msg = lazy AllowedMessage.fromSlice(in.body)) {  
    Transfer => { ... }  
    else => { ... }  
}
```

9.5 Lambdas and Lazy Loading

Lambda expressions have the form:

```
fun(x: int): int { return x + 1; }
```

Parameter types may be omitted in lambdas when inference can supply them.

The `lazy` operator delays deserialization and field loading:

```
val msg = lazy Transfer.fromSlice(in.body);  
val storage = lazy Storage.fromCell(contract.getData());
```

9.6 Operator Precedence

The parser implements the following precedence levels, from highest to lowest:

level	forms
100	primary expressions
90	function call and postfix !
80	dot access, method call, and numeric component access
75	unary !, ~, -, +
40	as, is, !is
30	*, /, %, ~/ , ^/
20	+, -
17	<<, >>, ~>>, ^>>
15	one comparison: ==, !=, <, >, <=, >=, <=>
14	bitwise &, , ^
13	logical &&,
10	assignment, compound assignment, ternary ?:, null coalescing ??

Comparison operators are not chainable. Assignment, ternary, and null coalescing are parsed right-to-left. The parser emits strict diagnostics for several ambiguous precedence patterns, such as mixing bitwise operators with comparisons without parentheses.

10 Contract Blocks

10.1 Purpose

The TOS actor-layer extension promotes the common message-dispatch pattern into compiler-visible syntax:

```
contract Name {
  storage: StorageType

  receive(msg: MessageType) {
    ...
  }

  get fun getter(): uint64 {
    ...
  }
}
```

}

The lowering synthesizes `loadData`, `save`, and `onInternalMessage`. If the contract declares external receivers, it also synthesizes `onExternalMessage`.

10.2 Contract Grammar

The accepted member forms are:

```
ContractDecl ::= [ ContractAnnotations ] "contract" name "{"
               ContractMember*
               "}"

ContractMember ::= "storage" ":" StorageType [ ";" ]
                  | "states" ":" State { "," State } [ ";" ]
                  | "@initial" "state" State [ ";" ]
                  | "@unknown_silent_drop" [ ";" ]
                  | "@unknown_throw" "(" integer-literal ")" [ ";" ]
                  | "@implicit_protocol_default" [ ";" ]
                  | "@implicit_protocol_for" "(" MessageType "," State ")" [ ";" ]
                  | [ "@deploy" ] [ "@disclaim_query_id" ] ReceiveBlock
                  | ReceiveExternalBlock
                  | [ GetterAnnotations ] "get" "fun" ...
```

The only contract annotation accepted before the `contract` keyword is `@on_bounced_policy("manual")` or `@on_bounced_policy("ignore")`. `StorageType` must be a simple type identifier, not a generic, union, nullable, or shaped type expression.

A contract block must contain exactly one `storage:` declaration and at least one internal `receive(...)` declaration. The storage declaration must appear before receivers and getters. Free-standing `fun` declarations are not permitted inside a contract block.

At the file level, the current parser supports one `contract` declaration per `.tol` file. A file containing a contract cannot also contain a top-level `onInternalMessage`. If the contract declares `receive_external`, the file cannot also contain a top-level `onExternalMessage`.

10.3 Typed Internal Receivers

Internal receivers have the form:

```
receive(msg: Transfer) {  
    ...  
}  
  
receive(msg: Transfer) on Open {  
    ...  
}
```

The message type must be a simple struct type with exactly a 32-bit serialization prefix. The parameter name is bound to a lazy decode of `in.body`. Normal receivers also have `storage` and `in` in scope.

The lowering dispatches on the first 32 bits of the body. It does not pre-load `queryId` as a global dispatch field; query-id semantics are receiver local.

Internal receiver opcodes must be unique. In a state-bearing contract, the same message struct and opcode may be used by multiple receivers only when each receiver has a distinct `on State` clause. A duplicate opcode/state pair, or a mix of state-qualified and unqualified receivers for the same opcode, is rejected.

10.4 Deployment Receivers

`@deploy` marks the bootstrap receiver:

```
@deploy  
receive(msg: Deploy) {  
    save(Storage { owner: msg.owner });  
}
```

The deploy branch runs before `loadData()` so that an empty `c4` can be initialized. Inside a deploy receiver, `storage` is not in scope. The receiver must materialize storage by calling `save(...)`.

`@deploy` is mutually exclusive with an `on State` clause and with `@disclaim_query_id`. At most one deploy receiver is allowed. In a state-bearing contract, a deploy receiver requires an `@initial state` declaration so that the compiler can inject the initial hidden state tag into the saved storage value.

10.5 Unknown Opcode Handling

There are four internal-message unknown-opcode modes:

- default mode: throw `OP_ERROR`, currently `0x00010001`;
- `@unknown_silent_drop`:: return normally without actions;
- `@unknown_throw(N)`:: throw the specified code;
- `receive(msg: UnknownOpcode) { ... }`: run a catch-all body with the raw `in.body` slice bound to the parameter.

The catch-all pseudo-type `UnknownOpcode` has no serialization encoding and cannot be used as an ordinary message body type. It is mutually exclusive with `@unknown_silent_drop` and `@unknown_throw`. If no visible unknown-opcode policy is declared, the compiler keeps the default Protocol throw but emits a receive-exhaustiveness warning. These unknown-opcode annotations apply to the internal-message dispatch domain only.

10.6 External Receivers

External receivers have the form:

```
receive_external(msg: SignedRequest) {  
    ...  
}
```

They synthesize `onExternalMessage` and dispatch on a 32-bit body prefix. Normal external receivers bind the parameter to a lazy decode of the external body and load `storage` before running the user body. They do not have internal-message `InMessage` context and do not participate in query-id propagation checks. State-machine guards apply only to internal receivers.

`receive_external(msg: UnknownOpcode) { ... }` is accepted as an external-only catch-all; its parameter is the raw `inMsgBody` slice. If no external catch-all is declared, an unrecognized external opcode throws the fixed Protocol sentinel $(4 \ll 16) \mid 0xffff$. The external catch-all does not synthesize the normal `storage` local. External opcodes must be unique within the external dispatch table, but they may collide with internal receiver opcodes because the entrypoints are separate.

10.7 Get Methods

Contract-local getters use:

```
get fun getBalance(): coins {  
    return storage.balance;  
}
```

```
@method_id(0x1001)  
get fun getOwner(): address {  
    return storage.owner;  
}
```

A `get fun` must declare an explicit return type. The compiler derives the method id from the getter name unless `@method_id(expr)` pins it. Method-id collisions are checked per contract.

Contract-local getters accept `@method_id`, `@inline`, `@inline_ref`, `@noinline`, `@pure`, and opaque custom annotations such as `@deprecated`. Getter bodies are read-only: calls that commit storage or emit actions, such as `save(...)`, `contract.setData(...)`, `sendRawMessage(...)`, `.send(...)`, or `.sendAndEstimateFee(...)`, are rejected. Getter names may not shadow generated contract internals such as `loadData`, `save`, `onInternalMessage`, or `onExternalMessage`.

11 State Machines

11.1 State Declaration

A state-bearing contract declares:

```
contract Auction {  
    storage: AuctionStorage  
    states: Open, Closed  
    @initial state Open  
  
    receive(msg: Bid) on Open {  
        ...  
        keep_state;  
    }  
}
```

```
    receive(msg: Close) on Open {  
        ...  
        become Closed;  
    }  
}
```

The compiler injects a hidden state tag into the serialized storage. User code may not declare or access a field named `__state`. A state-bearing contract must declare exactly one `@initial state`, and every ordinary internal receiver must declare an `on State` clause.

11.2 Transitions

Every control-flow path in a state-bearing receiver must end in either:

- `become State`, which writes a new hidden state tag;
- `keep_state`, which preserves the current state tag.

Both forms are tail-position statements. Code after either form is rejected as unreachable or invalid. `become` accepts only a static state identifier. `become` and `keep_state` are not valid inside `@deploy` receivers; deploy receivers use `@initial state` as the authoritative initial tag. They are also rejected inside `receive(msg: UnknownOpcode)` because the catch-all body has no source state.

11.3 State Checks

The compiler performs:

- state reachability checks from the initial state;
- receive exhaustiveness warnings over the declared message-by-state matrix;
- field-scoping checks for `@on(...)` storage fields;
- hidden state access checks.

An intentionally missing known-opcode/wrong-state pair may be documented with:

```
@implicit_protocol_for(MessageType, StateName);
```

or, for every missing pair:

```
@implicit_protocol_default;
```

12 Actor-Layer Static Analysis

12.1 Query-Id Propagation

TOS message policy treats `queryId` as the request/reply correlation field for envelope-shaped bodies. The compiler checks receiver scopes independently after contract lowering. A fire-and-forget receiver may state its intent explicitly:

```
@disclaim_query_id
receive(msg: BroadcastEvent) {
    ...
}
```

One receiver’s disclaimer does not silence warnings in a sibling receiver. The lower-level `disclaim_query_id()` builtin remains available in hand-written `onInternalMessage` code.

12.2 Require Code Assignment

`require(cond, ErrorClass.X)` is a standard-library/compiler surface used by receiver bodies. After contract lowering, the compiler assigns stable per-contract error codes to require sites without explicit third arguments and emits a require-site manifest comment in generated FIF output. Authors may pin a code explicitly:

```
require(amount <= storage.balance, ErrorClass.Permanent, 0x1201);
```

12.3 Storage and c4 Escape Restrictions

Inside `contract` receiver bodies, the compiler rejects several raw c4 escape paths that would bypass the declared storage and state model. Examples include direct `contract.getData()` access, wrappers that reach c4, and assembler bodies that mention c4/control-register aliases. Authors who need a raw migration path should use the lower-level hand-written entrypoint style instead of a `contract` block.

12.4 Postponement and Slice Checks

The Slice 4 postponement checker rejects unsafe postponement paths, including external-message enqueue attempts and direct queue internals that bypass the bounded `stdlib` helper. Slice 3 and Slice 5 checks cover reply-correlation and domain-pattern hardening surfaces used by the bundled standard library.

13 Standard Library Surface

13.1 Common Prelude

`@stdlib/common.tol` is loaded before the entrypoint file. It declares the built-in types, core TVM wrappers, arithmetic and logical helpers, builders, slices, cells, address helpers, message creation, send modes, reserve modes, random helpers, debugging helpers, and `contract/blockchain` namespaces.

The standard library presents many low-level FunC and TVM operations as typed methods:

```
cell.hash()
slice.loadUint(32)
builder.storeUint(value, 32)
contract.getData()
blockchain.now()
```

13.2 Explicit Imports

More specialized packages require explicit imports. The current repository contains, among others:

```
@stdlib/tvm-dicts
@stdlib/tvm-lowlevel
@stdlib/strings
@stdlib/gas-payments
@stdlib/reflection
@stdlib/exotic-cells
@stdlib/lisp-lists
@stdlib/slice3-common
@stdlib/jetton
@stdlib/nft
@stdlib/wallet
@stdlib/multisig
@stdlib/ownable
@stdlib/postponement
@stdlib/auction
@stdlib/governance
@stdlib/oracle
@stdlib/payment-channel
@stdlib/delivery
@stdlib/time
@stdlib/schedule
@stdlib/supervision
@stdlib/capability
@stdlib/safe-payments
```

Imports are explicit; a test or contract that uses symbols from a package directly must import that package, even if an imported production source also imports it.

13.3 Slice-Layer Packages

The TOS-specific stdlib is staged:

- Slice 1: envelope, error class, `OP_ERROR`, extra-flags constants, and query-id discipline;
- Slice 3: first-wave patterns for jetton, NFT, wallet, multisig, and ownable contracts;

- Slice 4: bounded postponement as explicit contract-owned storage;
- Slice 5: auction, governance, oracle, and payment-channel patterns plus ABI manifest validation;
- Slice 6: delivery records, scheduled-message shapes, time helpers, monitor/supervision helpers, capability grants, and safe-payment helpers.

Some Slice 6 protocol behavior remains activation-gated even though the repo-side Tol and stdlib surfaces are present.

14 Reserved but Unsupported Surface

The following tokens or forms are reserved or lexed but currently rejected:

- `export`, `operator`, and `infix` as top-level declarations;
- `break` and `continue`;
- `++` and `--`;
- multiple `contract` declarations in one `.tol` file;
- a `contract` block mixed with top-level `onInternalMessage`;
- free-standing `fun` declarations inside a contract block;
- receiver message prefixes that are not exactly 32 bits;
- `receive_external(...)` on `State`;
- `@deploy` combined with `on State` or `@disclaim_query_id`.

A Summary Grammar

This appendix collects the main grammar fragments in one place.

Source ::= { SourceItem }

SourceItem ::= TolVersion
 | Import
 | Annotation
 | GlobalDecl
 | ConstDecl
 | TypeAlias
 | FunctionDecl
 | StructDecl
 | EnumDecl
 | ContractDecl
 | TopLevelGetter
 | ";"

TolVersion ::= "tol" semver

Import ::= "import" string-literal

GlobalDecl ::= "global" identifier ":" TypeExpr

ConstDecl ::= "const" identifier [":" TypeExpr] "=" Expr

TypeAlias ::= "type" identifier [Generics] "=" (TypeExpr | "builtin")

FunctionDecl ::= "fun" [TypeExpr "."] identifier [Generics]
 ParameterList [":" (TypeExpr | "self")]
 (Block | AsmBody | "builtin")

AsmBody ::= "asm" ["(" { identifier | "self" }
 ["->" { integer-literal }] ")"]
 string-literal { string-literal }

TopLevelGetter ::= "get" "fun" identifier ParameterList
 [":" TypeExpr] Block

Generics ::= "<" GenericItem { "," GenericItem } [","] ">"

GenericItem ::= identifier ["=" TypeExpr]

ParameterList ::= "(" [Parameter { "," Parameter } [","]] ")"

Parameter ::= ["mutate"] (identifier | "self" | "_")

```

[ ":" TypeExpr ] [ "=" Expr ]

StructDecl ::= "struct" [ "(" Expr ")" ] identifier [ Generics ]
            [ "{" { StructField [ "," | ";" ] } "}" ]

StructField ::= [ Annotation* ] [ "private" ] [ "readonly" ]
              identifier ":" TypeExpr [ Annotation* ] [ "=" Expr ]

EnumDecl ::= "enum" identifier [ ":" TypeExpr ]
            "{" { EnumMember [ "," | ";" ] } "}"
EnumMember ::= identifier [ "=" Expr ]

ContractDecl ::= [ "@on_bounced_policy" "(" string-literal ")" ]
                "contract" identifier "{" { ContractMember } "}"

ContractMember ::= "storage" ":" identifier [ ";" ]
                  | "states" ":" identifier { "," identifier } [ ";" ]
                  | "@initial" "state" identifier [ ";" ]
                  | "@unknown_silent_drop" [ ";" ]
                  | "@unknown_throw" "(" integer-literal ")" [ ";" ]
                  | "@implicit_protocol_default" [ ";" ]
                  | "@implicit_protocol_for" "(" identifier "," identifier ")" [ ";" ]
                  | [ "@deploy" ] [ "@disclaim_query_id" ] ReceiveBlock
                  | ReceiveExternalBlock
                  | [ Annotation* ] "get" "fun" identifier ParameterList
                    ":" TypeExpr Block

ReceiveBlock ::= "receive" "(" identifier ":" identifier ")"
               [ "on" identifier ] Block

ReceiveExternalBlock ::= "receive_external"
                       "(" identifier ":" identifier ")" Block

TypeExpr ::= [ "|" ] NullableType { "|" NullableType } [ "->" TypeExpr ]
NullableType ::= SimpleType [ "<" TypeExpr { "," TypeExpr } [ "," ] ">" ]
               [ "?" ]

SimpleType ::= identifier
            | "self"

```

```
| "contract" | "receive" | "receive_external" | "storage"
| "null"
| "(" [ TypeExpr { "," TypeExpr } [ "," ] ] ")"
| "[" [ TypeExpr { "," TypeExpr } [ "," ] ] "]"
```

B Expression Precedence Summary

```
100 primary expressions
90 function call, postfix !
80 dot access, method call, numeric component access
75 unary ! ~ - +
40 as is !is
30 * / % ~/ ^/
20 + -
17 << >> ~>> ^>>
15 == != < > <= >= <=>
14 & | ^
13 && ||
10 = += -= *= /= %= <<= >>= &= |= ^= ?: ??
```

C Reference Sources

The implementation facts in this document are taken from the local repository:

- tol/lexer.cpp
- tol/ast-from-tokens.cpp
- tol/pipe-lower-contract.cpp
- tol/tol.cpp
- tol/pipeline.h
- crypto/smartcont/tol-stdlib/common.tol
- crypto/smartcont/tol-stdlib/*.tol
- doc/tos-language-syntax-policy.md